DOCUMENT

22

# Reviewer Guide

What to open, how to check every claim, what was decided and what is still open

This guide is for someone who has to judge the work and has not seen the project before. It tells you what was asked, what was built, what to open first, and how to check each claim yourself instead of taking it on trust.

## 1. The assignment in one paragraph

Take one small feature, **role-based invoice approval limits**, from specification to a reviewed pull request, and keep the evidence: a context file for AI agents, skills, hooks that log the AI work, a specification, staged implementation, meaningful unit tests (including the two negative rules), one browser test, mutation scores before and after, a review of the pull request with a verdict on every finding, a log of all AI interaction, a write-up, and a demo video.

The feature: a Manager may approve up to 50,000.00; a Director up to 500,000.00; above that two **different** approvers are needed and the second must be a Director. A refused attempt returns a clear reason and changes nothing.

## 2. What to open, in this order (about ten minutes)

| # | Open | You will see | Time |
|---|---|---|---|
| 1 | `docs/presentations/Invoice-Approval-Reviewer-Walkthrough.pdf` | The whole project in 35 slides | 5 min |
| 2 | `ASSIGNMENT.md` | The write-up: items met, mutation scores, decisions | 3 min |
| 3 | `specs/001-invoice-approval-limits/spec.md` | The rules as agreed, with the clarifications | 2 min |
| 4 | `docs/pr-review.md` | Eleven findings, each with one verdict | 3 min |
| 5 | `docs/presentations/Invoice-Approval-UI-Walkthrough.pdf` | The running application, screen by screen | 3 min |

## 3. Claims and how to check each one

| Claim | Evidence | How to check it yourself |
|---|---|---|
| The limits are inclusive and tested one cent either side | `test_approval_policy.py`, `test_approval_service.py` | Run the unit tests; search for `50000.01` and `500000.01` |
| The same user cannot approve twice, even as `Dir.One` versus `dir.one` | policy, service and API tests | `python -m pytest backend/tests/test_approval_policy.py -v -k "same_user"` |

| Claim | Evidence | How to check it yourself |
|---|---|---|
| A Manager cannot be the second approver above 500,000 | the same files | `-k "second_approver"` |
| A refused attempt changes nothing | the helper `assert_refused_and_unchanged` in the service tests: deep copy before, expect the error, compare after, assert the reason text, assert the repository saw no update | Read the helper; then break the code (see section 5) and watch the tests fail |
| No partial writes under concurrency | `ApprovalService` holds one lock; threaded tests use a database-like repository that returns snapshots | The tests fail (2 failed) if the lock is replaced by a no-op |
| The tests can actually fail | Mutation score 63.9% before, 100% after (132 of 132) | `wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh check` (about 13 s) |
| The UI shows the reason and keeps the invoice pending | `e2e/tests/over-limit-rejection.spec.ts` | `cd e2e; npm install; npx playwright install chromium; npm test` |
| Every AI interaction is logged in one fixed format | `logs/ai-interactions.jsonl` : 27 keys per line | `python scripts/summarize_log.py` validates every line |
| Every review finding has exactly one verdict | `docs/pr-review.md` | Count: 6 fixed, 3 dismissed, 2 deferred = 11 |

## 4. Repository map

```
AGENTS.md  CLAUDE.md          Context file for AI agents (CLAUDE.md imports AGENTS.md)
ASSIGNMENT.md                 The write-up, with scores, decisions and the video link
backend/src/invoice_app/
    domain/                   Business rules (no web, no storage code)
    api/                      FastAPI app, routers, sign-in, in-memory repository, web UI
(wwwroot/)
backend/tests/                177 tests, one module per module under test
e2e/                          Playwright test, page objects
specs/001-invoice-approval-limits/   Spec Kit specification, plan, research, data model,
contract, tasks
.specify/                     Spec Kit scaffold and constitution
.claude/                      Agents, skills, hooks, settings
logs/                         AI interaction log and its readable summary
docs/guides/                  The documentation (Markdown); docs/pdf/ has it as PDFs
docs/mutation/                Mutation reports (before, after)
docs/presentations/           Slide decks (PowerPoint and PDF)
scripts/                      Mutation helpers, PDF/diagram/deck builders, log summary,
try_all_cases.py
.github/workflows/ci.yml      Runs the tests on every push
```

## 5. See a test fail on purpose (two minutes)

This is the quickest way to believe that the tests mean something.

1. Open `backend/src/invoice_app/domain/approval_policy.py` and find the first comparison in `ManagerLimitRule.refusal` (`self._limits.manager < attempt.amount <= self._limits.director`).
2. Change the first `<` to `<=`. A Manager would now be refused at exactly 50,000.00.
3. Run `python -m pytest backend/tests -q -x`. The run stops at the first failing test and names the boundary.
4. Undo the change.

The same idea is what `mutmut` does automatically, 132 times.

## 6. What was decided where there was ambiguity

These were written down in the specification (section "Clarifications") before any code:

| Question | Decision |
|---|---|
| Are the limits inclusive? | Yes. 50,000.00 is allowed for a Manager; 50,000.01 needs a Director. Likewise 500,000.00 and 500,000.01 |

| Question | Decision |
| --- | --- |
| Who may give the first approval above 500,000? | A Manager or a Director. Only the second approver is constrained |
| What is "the same user"? | The same account after trimming spaces and ignoring case |
| Which message wins when the same Manager tries twice? | The duplicate-user reason (the more specific problem) |
| What happens to the old `Approver` role? | Replaced by `Manager`. `Admin` can reject and delete but never approve |
| What is "a rejected attempt"? | An approval the rules refuse (403 with a reason), not the Reject button |
| Does a refused attempt undo a first approval? | A refused attempt never writes anything; an accepted first approval stays |

## 7. Questions a reviewer is likely to ask

**Why is the app in Python when the original is C#?** The task asked for Python and the machine had no .NET SDK. The backend was ported to FastAPI with the same layering, endpoints, web UI and Playwright setup; the seven original tests were ported first and kept green before any feature work.

**Why mutation testing and not coverage?** Coverage says a line ran; it does not say a test would notice if the line were wrong. The first pass after the feature scored 81.2% with 16 survivors, all missing tests (for example a fake repository whose `update()` did nothing, so "forgot to save" passed). Closing them reached 100%.

**Why did the mutant count change from 36 to 132?** The scope is the whole domain package in every run; new files (the approval policy and approval records) added mutants. Compare the kill rate, not the counts.

**What did the AI get wrong?** A concurrency test that passed even with the lock removed (it proved nothing and was rewritten), a hook that read `passed=82 failed=0` as 82 failed, log lines interleaved by parallel agents, and a first "control" run in the context experiment that was not a control. All are recorded in `ASSIGNMENT.md` and the log.

**What is not done?** Two findings are deferred and tracked (T024: a second browser test and a reset endpoint; T025: session expiry and constant-time password comparison). The demo's sign-in is not production authentication.

## 8. Commands in one place

```
pip install -e "backend[dev]"
python -m pytest backend/tests -q
python -m pytest .claude/hooks -q
python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000
cd e2e && npm install && npx playwright install chromium && npm test
wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh check
python scripts/summarize_log.py
```

Demo accounts (local only): `demo.manager`, `demo.director`, `demo.director2`, `demo.admin`, password `ChangeMe123!`. Try `INV-2001` (75,000.00, needs a Director) and `INV-2002` (600,000.00, needs two approvers).

## 9. Other documents

The numbered guides in this set go deeper: 02 Requirements, 03 Architecture, 04 API, 07 Test Plan, 08 Mutation, 09 Code Review, 10 AI Workflow, 19 SOLID, 20 Code Reference, 21 Architecture Diagrams, 23 UI Walkthrough.